

```
In [1]: from notebook_native_agent import start_agent, stop_agent, agent_status, agent_symbols, pause_agent, resume_agent, get_state
start_agent()
```

⚡ Notebook agent active

provider: openai

auto-execute: on

fallback: on

LLM cell classifier: off

Notebook Native Agent

Write Python normally, or write a natural-language request in a cell. The agent shares the live notebook context, proposes code, and can safely auto-run allowed cases.

Try a natural-language cell like:

show me a funny fractal

copy

plot this: $z = \sin(x) * \cos(y)$ (use plotly)

copy

I have this: tips.csv

copy

How it works

1 Stay in the notebook — write Python or natural-language requests directly in code cells; the router turns non-Python text into agent requests.

2 Work with live context — the agent tracks notebook symbols, reusable helpers, object summaries, and prior executions.

3 Preview or auto-run — generated code is shown, safety-checked, tracked, and auto-run only when allowed.

Fallback mode

Demo fallback is enabled. If no brain model/API key is available, the agent shows one fixed safe example: a graph of the sine function.

```
In [2]: Fa un graf cu orașe și rute, si apoi afiseaza cele mai scurte rute dintr-un oraș
```

Notebook Native Agent

What I understood

Creez un graf cu orașe și rute și afișez cele mai scurte drumuri dintr-un oraș ales.

Proposed solution

Voi construi un exemplu de rețea de orașe și rute ponderate cu distanțe, apoi:

1. creez graful `cities_graph` din `routes_df`
2. desenez graful și evidențiez orașul de pornire `start_city`
3. calculez cele mai scurte rute către toate celelalte orașe
4. afișez rezultatul într-un tabel `shortest_paths_df`

Presupunere vizibilă: fiindcă orașul de start nu a fost specificat, folosesc implicit `start_city = "București"`, pe care îl poți schimba ușor ulterior.

Simboluri reutilizabile create: `routes_df`, `cities_graph`, `start_city`, `shortest_paths_df`.

Code preview

[copy](#)

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
from IPython.display import display

def build_city_graph(routes_df: pd.DataFrame) -> nx.Graph:
    """Build an undirected weighted graph from a routes dataframe."""
    graph = nx.Graph()
    for _, row in routes_df.iterrows():
        graph.add_edge(row["source"], row["target"], distance=row["distance_km"])
    return graph

def compute_shortest_routes(graph: nx.Graph, start_city: str) -> pd.DataFrame:
    """Compute shortest weighted routes from the selected start city."""
    if start_city not in graph:
        raise ValueError(f'Orașul '{start_city}' nu există în graf.')

    path_lengths = nx.single_source_dijkstra_path_length(graph, start_city, weight="distance")
    paths = nx.single_source_dijkstra_path(graph, start_city, weight="distance")

    rows = []
    for city, distance in path_lengths.items():
        rows.append(
            {
                "destination": city,
                "distance_km": distance,
                "path": " -> ".join(paths[city]),
                "stops": max(len(paths[city]) - 1, 0),
            }
        )

    shortest_paths_df = pd.DataFrame(rows).sort_values(["distance_km", "destination"]).reset_index(drop=True)
    return shortest_paths_df

def plot_city_graph(graph: nx.Graph, start_city: str):
    """Plot the city graph and highlight the start city."""
    fig, ax = plt.subplots(figsize=(10, 7))
    pos = nx.spring_layout(graph, seed=42)

    node_colors = ["tomato" if node == start_city else "skyblue" for node in graph.nodes()]
    edge_labels = nx.get_edge_attributes(graph, "distance")

    nx.draw_networkx(
        graph,
        pos=pos,
        ax=ax,
        with_labels=True,
        node_color=node_colors,
        node_size=1800,
        font_size=10,
        edge_color="gray",
        width=2,
    )
    nx.draw_networkx_edge_labels(
        graph,
        pos=pos,
        edge_labels={edge: f'({dist}) km' for edge, dist in edge_labels.items()},
        font_size=9,
        ax=ax,
```

```
)

ax.set_title(f"Graf orașe și rute (start: {start_city})")
ax.axis("off")
return fig

routes_data = [
    ("București", "Ploiești", 60),
    ("București", "Brașov", 170),
    ("București", "Constanța", 225),
    ("Ploiești", "Brașov", 110),
    ("Ploiești", "Buzău", 95),
    ("Buzău", "Iași", 310),
    ("Brașov", "Sibiu", 145),
    ("Sibiu", "Cluj-Napoca", 170),
    ("Cluj-Napoca", "Oradea", 155),
    ("Brașov", "Târgu Mureș", 165),
    ("Târgu Mureș", "Cluj-Napoca", 100),
    ("Constanța", "Galați", 190),
    ("Galați", "Iași", 230),
    ("București", "Craiova", 210),
    ("Craiova", "Sibiu", 175),
    ("Craiova", "Timișoara", 290),
    ("Timișoara", "Oradea", 170),
]

routes_df = pd.DataFrame(routes_data, columns=["source", "target", "distance_km"])
cities_graph = build_city_graph(routes_df)
start_city = "București"
shortest_paths_df = compute_shortest_routes(cities_graph, start_city)

print("Rute disponibile:")
display(routes_df)

fig = plot_city_graph(cities_graph, start_city)
display(fig)
plt.close(fig)

print(f"Cele mai scurte rute din {start_city}:")
display(shortest_paths_df)
```

Execution

auto-execute via gpt-5.4

Selected context

No prior notebook context looked necessary.

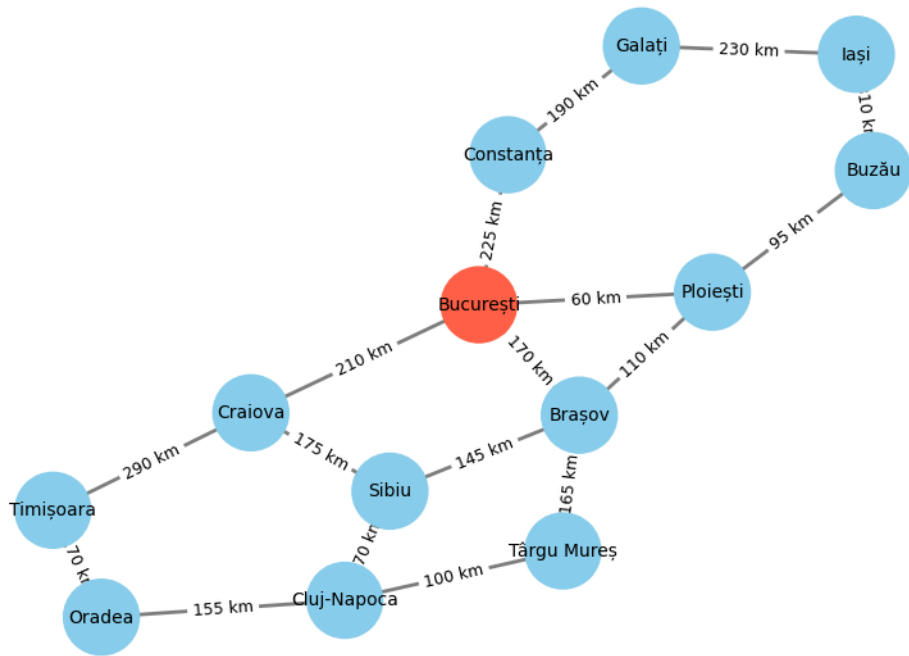
Safety notes

- Looks like an in-memory inspection or plotting task.

Rute disponibile:

	source	target	distance_km
0	București	Ploiești	60
1	București	Brașov	170
2	București	Constanța	225
3	Ploiești	Brașov	110
4	Ploiești	Buzău	95
5	Buzău	Iași	310
6	Brașov	Sibiu	145
7	Sibiu	Cluj-Napoca	170
8	Cluj-Napoca	Oradea	155
9	Brașov	Târgu Mureș	165
10	Târgu Mureș	Cluj-Napoca	100
11	Constanța	Galați	190
12	Galați	Iași	230
13	București	Craiova	210
14	Craiova	Sibiu	175
15	Craiova	Timișoara	290
16	Timișoara	Oradea	170

Graf orașe și rute (start: București)



Cele mai scurte rute din București:

	destination	distance_km	path	stops
0	București	0	București	0
1	Ploiești	60	București -> Ploiești	1
2	Buzău	155	București -> Ploiești -> Buzău	2
3	Brașov	170	București -> Brașov	1
4	Craiova	210	București -> Craiova	1
5	Constanța	225	București -> Constanța	1
6	Sibiu	315	București -> Brașov -> Sibiu	2
7	Târgu Mureș	335	București -> Brașov -> Târgu Mureș	2
8	Galați	415	București -> Constanța -> Galați	2
9	Cluj-Napoca	435	București -> Brașov -> Târgu Mureș -> Cluj-Napoca	3
10	Iași	465	București -> Ploiești -> Buzău -> Iași	3
11	Timișoara	500	București -> Craiova -> Timișoara	2
12	Oradea	590	București -> Brașov -> Târgu Mureș -> Cluj-Napoca -> Oradea	4

```
In [ ]:
```